



Proyectos de programación

Estructuras de Datos y Algoritmos

“Extractor de prototipus de comportament o perfils”

Arnau Pons Cardet (arnau.pons.cardet@estudiantat.upc.edu)

Felipe Núñez Gómez (felipe.arturo.nunez@estudiantat.upc.edu)

Yeray Llagostera Garcia (yeray.llagostera@estudiantat.upc.edu)

Kiara Kristin Rodriguez Uribe (kiara.kristin.rodriiguez@estudiantat.upc.edu)

Víctor Huertes Montes (victor.huertes@estudiantat.upc.edu)

Grup 32.1

Versió 1.0

Ignasi Gómez-Sebastià

2025/2026 Q1

ÍNDICE:

1. Estructuras de Datos y Algoritmos.....	2
1.1. Estructuras de Datos.....	2
1.2. Algoritmos.....	3
Agrupamiento K-Means.....	3
Agrupamiento K-Means++.....	5
Agrupamiento K-Medoids.....	8

1. Estructuras de Datos y Algoritmos

1.1. Estructuras de Datos

Las estructuras de datos se organizan en tres repositorios claves, diseñados para ofrecer un acceso eficiente a la información principal (usuarios, encuestas y respuestas) basándose en los casos de uso principales de la aplicación.

4.1.1 Repositorio de Usuarios(UserRepository)

El repositorio almacena la información mediante un `Map<String, User> users`, indexado por el 'username' de cada User que actúa como su identificador.

Cada usuario almacena dos estructuras para garantizar la eficiencia a los siguientes accesos principales a partir de un usuario:

1. Encuestas creadas: La estructura `Set<String> createdSurveysId`, permite un acceso directo a cada encuesta creada de un usuario.
2. Respuestas propias. La estructura `Map<String, Set<String>> responsesBySurveyId`, indexado por el 'surveyId' (identificador de una encuesta), permite acceder de manera directa a cada respuesta de un usuario a cada encuesta mediante el set de 'responsId', identificador de una respuesta.

4.1.2 Repositorio de Encuestas(SurveyRepository)

El repositorio almacena la información mediante un `Map<String, Survey> surveys`, indexado por el surveyId de cada encuesta que actúa como identificador.

Cada encuesta guarda el username de su creador, el cuál es único, mediante el atributo `CREATOR_USERNAME`.

Además, almacena todas las preguntas mediante un `List<Question> questions`.

Cada una de estas preguntas, mediante el atributo `SURVEY_ID`, guarda la encuesta a la que pertenece y la posición que ocupan en la encuesta a través del atributo `questionIndex`.

4.1.3 Repositorio de Respuestas(ResponseRepository)

El repositorio almacena la información mediante un `Map<String, Map<String, Response>> responses`. El índice externo es el identificador de una encuesta(surveyId) y el interno es el identificador de una respuesta(responsId).

Este anidamiento de índices se utiliza para mejorar la eficiencia del acceso a las respuestas a partir de una encuesta, que en nuestra aplicación es siempre.

Cada respuesta guardan su identificador(responsId), el identificador a la encuesta a la que pertenece(surveyId), y el identificador el usuario que responde(`RESPONDER_USERNAME`).

Además almacena todas las réplicas a cada una de las preguntas, estas guardan el identificador a la respuesta a la que pertenecen(`RESPONSE_ID`) y la posición de la pregunta a la que contestan(`QUESTION_INDEX`).

1.2. Algoritmos

Esta parte del documento proporciona una explicación detallada del algoritmo K-Means, K-Means++ y K-Medoids tal como se implementa en este proyecto.

Agrupamiento K-Means

1. Justificación e Introducción

K-Means es un popular algoritmo de aprendizaje automático no supervisado que se utiliza para dividir un conjunto de datos en un conjunto de 'k' clústeres. El algoritmo asigna iterativamente cada punto de datos al clúster más cercano y luego recalcula el centro del clúster (centroide). Este proceso se repite hasta que las asignaciones de clúster ya no cambian, o se alcanza un número máximo de iteraciones.

En el contexto de esta aplicación, K-Means se utiliza para agrupar respuestas de encuestas similares. Esto permite descubrir patrones y segmentos dentro de la población de encuestados. Por ejemplo, puede ayudar a identificar grupos de usuarios con preferencias u opiniones similares en función de sus respuestas.

2. Pseudocódigo del Algoritmo K-Means Genérico

1. **Inicialización:** Seleccionar aleatoriamente 'k' puntos de datos del conjunto de datos para que sean los centroides iniciales.
2. **Paso de Asignación:** Para cada punto de datos en el conjunto de datos, calcular la distancia a cada uno de los 'k' centroides. Asignar el punto de datos al clúster del centroide más cercano.
3. **Paso de Actualización:** Para cada clúster, recalculamos el centroide tomando la media de todos los puntos de datos asignados a ese clúster.
4. **Verificación de Convergencia:** Repetir los pasos de Asignación y Actualización hasta que los centroides ya no se muevan significativamente entre iteraciones, o se alcance un número máximo de iteraciones.

3. Justificación de Cambios Clave Respecto al Algoritmo Genérico

Esta aplicación maneja respuestas de encuestas que pueden contener una mezcla de preguntas numéricas, textuales y de opción múltiple. El algoritmo K-Means genérico está diseñado para datos numéricos y no puede manejar esta variedad de tipos de datos. Por lo tanto, nuestra implementación introduce varias modificaciones clave para abordar este desafío.

Enfoque Híbrido Centroide/Medoides

La principal innovación de nuestra implementación es un enfoque híbrido que combina centroides para datos numéricos y de opción múltiple con medoides para datos textuales.

- **Preguntas Numéricas y de Opción Múltiple:** Para estos tipos de preguntas, la implementación calcula un verdadero centroide.

- Numéricas: El valor del centroide es la media de todas las respuestas numéricas en el clúster.
- Opción Múltiple: Las respuestas se representan como vectores booleanos, que se convierten a `double[]` (1.0 para seleccionado, 0.0 para no seleccionado). El centroide es la media de estos vectores.
- **Preguntas Textuales**: Para las respuestas textuales, calcular una "media" no es factible. En su lugar, la implementación encuentra un medoide. El medoide es una respuesta real del clúster que es la más representativa de todas las demás respuestas en ese clúster. Es el punto de datos con la distancia promedio mínima a todos los demás puntos de datos en el mismo clúster.

Este enfoque híbrido permite que el algoritmo maneje tipos de datos heterogéneos de manera efectiva, proporcionando un agrupamiento más preciso y significativo de las respuestas de la encuesta.

Métricas de Distancia Personalizadas

Para admitir el modelo de datos híbrido, hemos implementado un [DistanceCalculator](#) flexible que utiliza diferentes métricas de distancia para diferentes tipos de preguntas:

- Numéricas: La distancia es la diferencia absoluta entre el valor de la respuesta y el valor del centroide.
- Opción Múltiple: La distancia es la distancia euclidiana entre el vector de la respuesta y el vector del centroide.
- Textuales: La distancia es la distancia de Levenshtein normalizada, que mide la diferencia entre two cadenas de texto.

El [DistanceCalculator](#) también admite ponderación, lo que permite que ciertas preguntas tengan una mayor influencia en el proceso de agrupamiento.

4. Correctitud

La correctitud de nuestra implementación de K-Means está garantizada por lo siguiente:

- Adhesión a los Principios de K-Means: El algoritmo sigue el proceso iterativo fundamental de los pasos de asignación y actualización, que es el núcleo del algoritmo K-Means.
- Manejo Sólido de Tipos de Datos: El uso de estructuras de datos apropiadas (centroides y medoides) y métricas de distancia para cada tipo de dato garantiza que el agrupamiento se realice de manera significativa.
- Convergencia: El algoritmo incluye una verificación de convergencia basada en el movimiento de los centroides, lo que garantiza que el proceso terminará.
- Pruebas Unitarias: La implementación está acompañada de un conjunto de pruebas unitarias en [src/test/java/clustering/TestKMeans.java](#). Estas pruebas verifican la correctitud de los componentes del algoritmo, incluidos los cálculos de distancia y el proceso de agrupamiento general.

5. Uso de Tiempo y Memoria

Sea:

- n el número de respuestas.
- k el número de clústeres.
- d el número de preguntas (dimensiones).

- i el número de iteraciones.

Complejidad Temporal

La complejidad temporal general del algoritmo es $O(n * k * d * i)$. Esto se puede desglosar de la siguiente manera:

- Inicialización: $O(k * d)$ - Seleccionar 'k' centroides iniciales.
- Paso de Asignación: $O(n * k * d)$ - Para cada una de las 'n' respuestas, calculamos la distancia a 'k' centroides, y cada cálculo de distancia implica iterar sobre 'd' dimensiones.
- Paso de Actualización: $O(n * d)$ - Iteramos a través de todas las 'n' respuestas y sus 'd' dimensiones para recalculamos los centroides. El cálculo del medoide para datos textuales es más costoso, pero en el contexto del algoritmo general, su impacto se absorbe en la complejidad $O(n*d)$ del paso de actualización.

Dado que el paso de asignación es la operación más costosa y se realiza en cada iteración, la complejidad temporal total está dominada por $O(n * k * d * i)$.

Complejidad de Memoria

La complejidad de memoria del algoritmo es $O(n * d + k * d)$.

- $O(n * d)$: Para almacenar el conjunto de datos completo de 'n' respuestas, cada una con 'd' dimensiones.
- $O(k * d)$: Para almacenar los 'k' centroides, cada uno con 'd' dimensiones.

El uso de memoria está impulsado principalmente por el tamaño del conjunto de datos y el número de clústeres.

Agrupamiento K-Means++

1. Justificación e Introducción

K-Means++ es una versión mejorada del algoritmo K-Means estándar. Su principal mejora radica en la inicialización de los centroides de los clústeres. Mientras que el algoritmo K-Means estándar elige los centroides iniciales de forma aleatoria, K-Means++ utiliza un enfoque probabilístico "más inteligente". Este método distribuye los centroides iniciales, lo que ayuda a evitar resultados de agrupamiento deficientes que pueden ocurrir cuando los centroides iniciales se eligen cerca unos de otros. El resultado es un tiempo de convergencia más rápido y una menor probabilidad de encontrar un agrupamiento subóptimo.

En esta aplicación,, K-Means++ se ofrece como una alternativa más robusta al algoritmo K-Means estándar para agrupar las respuestas de las encuestas.

2. Pseudocódigo del Algoritmo de Inicialización K-Means++

1. **Elegir el Primer Centroide:** Seleccionar un punto de datos de forma uniforme y aleatoria del conjunto de datos y establecerlo como el primer centroide.
2. **Elegir los Centroides Sigüientes:** Para cada punto de datos x en el conjunto de datos, calcular $D(x)$, la distancia entre x y el centroide más cercano que ya ha sido elegido.

3. **Seleccionar el siguiente centroide** de los puntos de datos restantes con una probabilidad proporcional a $D(x)^2$. Esto significa que los puntos de datos más alejados de los centroides existentes tienen más probabilidades de ser elegidos como el siguiente centroide.
4. **Repetir:** Repetir los pasos 2 y 3 hasta que se hayan elegido 'k' centroides.
5. **Proceder con K-Means Estándar:** Una vez que se seleccionan los 'k' centroides iniciales, proceder con el algoritmo K-Means estándar (pasos de asignación y actualización) hasta la convergencia.

3. Justificación de Cambios Clave Respecto al Algoritmo Genérico

La implementación de K-Means++ en este proyecto se basa en la implementación híbrida de K-Means existente. Reutiliza la misma lógica central para manejar tipos de datos mixtos, pero reemplaza la inicialización aleatoria con el método K-Means++.

Inicialización Inteligente para Datos Híbridos

El método `initializeCentroidsPlusPlus` es el núcleo de esta implementación. Adapta la inicialización de K-Means++ para nuestro modelo de datos híbrido:

- Comienza seleccionando una respuesta al azar para que sea el primer centroide.
- Luego, para cada centroide posterior, calcula la distancia al cuadrado de cada respuesta a su centroide existente más cercano utilizando el `DistanceCalculator` personalizado. Esta calculadora maneja correctamente la mezcla de preguntas numéricas, textuales y de opción múltiple.
- A continuación, selecciona un nuevo centroide de las respuestas, donde cada respuesta tiene una probabilidad de ser elegida proporcional a su distancia al cuadrado calculada. Esta selección probabilística es la clave para distribuir los centroides iniciales.

Al integrar la inicialización de K-Means++ con nuestro `DistanceCalculator` existente, obtenemos los beneficios de una inicialización más inteligente sin dejar de admitir plenamente la naturaleza compleja y de tipos de datos mixtos de las respuestas de la encuesta.

Reutilización de Código

Después del paso de inicialización, la implementación de `KMeansPlusPlus.java` llama a los mismos métodos `assignToClusters`, `updateCentroids` y `hasConverged` que la implementación estándar de `KMeans.java`. Este es un uso eficiente del código que evita la duplicación y garantiza que la lógica de agrupamiento central se mantenga consistente entre los dos algoritmos.

4. Correctitud

La correctitud de nuestra implementación de K-Means++ se basa en:

- **Implementación Correcta de la Inicialización:** El método `initializeCentroidsPlusPlus` implementa correctamente la selección probabilística de centroides, que es la característica definitoria del algoritmo K-Means++.

- **Aprovechamiento de una Implementación Correcta de K-Means:** Después de la inicialización, el algoritmo se basa en la misma lógica central que nuestra implementación estándar de [K-Means](#), que ha sido verificada en cuanto a su correctitud.
- **Pruebas Unitarias:** La implementación es validada por un conjunto de pruebas dedicado en [src/test/java/clustering/TestKMeansPlusPlus.java](#). Estas pruebas aseguran que la inicialización funcione como se espera y que el algoritmo general produzca resultados correctos.

5. Uso de Tiempo y Memoria

Sea:

- **n** el número de respuestas.
- **k** el número de clústeres.
- **d** el número de preguntas (dimensiones).
- **i** el número de iteraciones.

Complejidad Temporal

La complejidad temporal de K-Means++ está dominada por su inicialización y las posteriores iteraciones de K-Means.

- **Inicialización:** $O(n * k * d)$ - La inicialización de K-Means++ es computacionalmente más costosa que la inicialización aleatoria de K-Means estándar. Para elegir cada uno de los 'k' centroides, necesitamos calcular la distancia desde cada uno de los 'n' puntos a los centroides ya elegidos. Esto da como resultado una complejidad temporal de $O(n * k * d)$ para la fase de inicialización.
- **Iteraciones de K-Means:** $O(n * k * d * i)$ - La complejidad de la parte iterativa del algoritmo es la misma que la de K-Means estándar.

Aunque la inicialización es más lenta, K-Means++ a menudo conduce a un menor número de iteraciones (**i**) necesarias para la convergencia, lo que puede hacerlo más rápido en general en la práctica.

Complejidad de Memoria

La complejidad de memoria es $O(n * d + k * d)$, la misma que la del algoritmo K-Means estándar.

- **$O(n * d)$:** Para almacenar el conjunto de datos.
- **$O(k * d)$:** Para almacenar los centroides.
- **$O(n)$:** Se requiere un $O(n)$ adicional durante la inicialización para almacenar las distancias al cuadrado de cada punto a su centroide más cercano, pero esto no cambia la complejidad de memoria general.

Agrupamiento K-Medoids

1. Justificación e Introducción

K-Medoids es un algoritmo de agrupamiento similar a K-Means. Sin embargo, en lugar de representar el centro del clúster con una media calculada (un centroide), K-Medoids utiliza un punto de datos real del clúster, conocido como medoide. El medoide es el punto dentro de un clúster que tiene la disimilitud promedio mínima con todos los demás puntos del mismo clúster.

Las principales ventajas de K-Medoids sobre K-Means son:

- **Robustez a los Valores Atípicos (Outliers):** Dado que el medoide debe ser un punto de datos real, el centro del clúster se ve menos afectado por los valores atípicos que una media calculada.
- **Funciona con Distancias Arbitrarias:** K-Medoids se puede utilizar con cualquier medida de disimilitud, mientras que K-Means generalmente requiere una métrica de distancia que permita el cálculo de una media (como la distancia euclidiana).

En esta aplicación, K-Medoids ofrece una alternativa robusta a K-Means, especialmente cuando los datos pueden contener valores atípicos o cuando se desea un centro de clúster más interpretable (una respuesta de encuesta real).

2. Pseudocódigo del Algoritmo K-Medoids (PAM)

La implementación en este proyecto sigue los principios del algoritmo Partitioning Around Medoids (PAM).

- **Inicialización (BUILD):** Seleccionar aleatoriamente '*k*' puntos de datos del conjunto de datos para que sean los medoides iniciales.
- **Paso de Asignación:** Asignar cada punto de datos al clúster del medoide más cercano.
- **Paso de Actualización (SWAP):** Para cada clúster, iterar a través de todos los puntos de datos no medoides dentro de ese clúster. Considerar intercambiar el medoide actual con uno de los puntos no medoides. Si el intercambio resulta en una disminución del costo total (la suma de las distancias de todos los puntos a su nuevo medoide), entonces realizar el intercambio.
- **Verificación de Convergencia:** Repetir los pasos de Asignación y Actualización hasta que los medoides ya no cambien entre iteraciones, o se alcance un número máximo de iteraciones. En nuestra implementación, la convergencia se verifica observando si el cambio en el costo total está por debajo de una tolerancia dada.

3. Justificación de Cambios Clave Respecto al Algoritmo Genérico

Nuestra implementación de K-Medoids está adaptada para funcionar con las estructuras de datos y los requisitos específicos del proyecto.

Manejo de Datos Híbridos

Al igual que nuestras implementaciones de K-Means y K-Means++, el algoritmo K-Medoids está diseñado para funcionar con los datos híbridos de las respuestas de la encuesta. Esto se logra utilizando el mismo [DistanceCalculator](#) personalizado, que puede calcular distancias entre dos objetos [Response](#) que contienen una mezcla de respuestas numéricas, textuales y de opción múltiple. Esta es una característica clave, ya que permite aplicar K-Medoids directamente a nuestros datos complejos sin requerir ningún preprocesamiento o incrustación en un espacio puramente numérico.

Implementación del Paso de Actualización (Swap)

El método `updateMedoids` implementa la lógica central del paso SWAP. Para cada clúster, itera a través de cada punto miembro y lo considera como un medoide potencial. Calcula la distancia total intra-clúster para cada candidato. El punto que minimiza esta suma se elige como el nuevo medoide para ese clúster. Esto asegura que el nuevo medoide sea el punto más central y representativo dentro de su clúster.

Representación Consistente del Clúster

Aunque K-Medoids funciona con medoides (que son objetos [Response](#)), los objetos [Cluster](#) finales se crean con una representación [Centroid](#) del medoide. Esto se logra convirtiendo el [Response](#) medoide final en un objeto [Centroid](#). Esta elección de diseño asegura que la estructura de salida ([ClusterResults](#)) sea consistente en todos los algoritmos de agrupamiento del proyecto, simplificando el post-procesamiento y el análisis.

4. Correctitud

La correctitud de nuestra implementación de K-Medoids está garantizada por:

- **Adhesión a los Principios de PAM:** El algoritmo sigue correctamente el proceso iterativo de asignación e intercambio de medoides que define el algoritmo Partitioning Around Medoids.
- **Función de Costo:** El objetivo del algoritmo es minimizar una función de costo bien definida (la suma de las distancias de todos los puntos a su medoide), y el paso de actualización está diseñado explícitamente para lograr esto.
- **Convergencia:** El algoritmo tiene garantizada la convergencia porque el paso de actualización solo acepta intercambios que reducen el costo total, y hay un número finito de configuraciones de medoide posibles.
- **Pruebas Unitarias:** La implementación es validada por un conjunto de pruebas dedicado en [src/test/java/clustering/TestKMedoids.java](#). Estas pruebas verifican la correctitud de la lógica del algoritmo.

5. Uso de Tiempo y Memoria

Sea:

- **n** el número de respuestas.
- **k** el número de clústeres.
- **d** el número de preguntas (dimensiones).
- **i** el número de iteraciones.
- **n_c** el número de puntos en un clúster.

Complejidad Temporal

La complejidad temporal de K-Medoids es generalmente mayor que la de K-Means, principalmente debido al costoso paso de actualización. La complejidad general es $O(k * (n-k)^2 * d * i)$ en el peor de los casos para algunas implementaciones de PAM, pero nuestra implementación es más eficiente.

- **Inicialización:** $O(k)$ - Seleccionar 'k' índices aleatorios.
- **Paso de Asignación:** $O(n * k * d)$ - Para cada una de las 'n' respuestas, calcular la distancia a 'k' medoides.
- **Paso de Actualización:** $O(\text{Suma sobre todos los clústeres}(n_c^2 * d))$ que se simplifica a aproximadamente $O(n^2 * d / k)$. Para cada uno de los k clústeres, iteramos a través de cada uno de los n_c puntos como un medoide candidato, y para cada candidato, calculamos la suma de las distancias a todos los demás n_c puntos.

El paso de actualización es el cuello de botella. Dado que esto se realiza en cada iteración, la complejidad temporal total está dominada por el paso de actualización, lo que lo hace significativamente más lento que K-Means, especialmente para grandes conjuntos de datos.

Complejidad de Memoria

La complejidad de memoria es $O(n * d)$.

- **$O(n * d)$:** Para almacenar todo el conjunto de datos.
- **$O(k)$:** Para almacenar los índices de los 'k' medoides.

El uso de memoria está dominado por el almacenamiento del propio conjunto de datos.